

Client–Server Messaging System

Detailed Design Document

Introduction

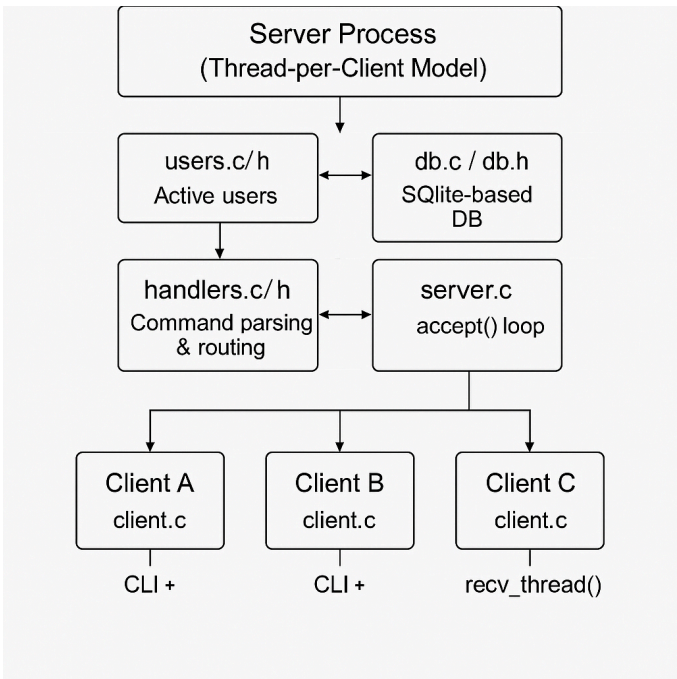
This document describes the detailed design of a multi-client messaging system built using a client–server architecture over TCP. The system supports:

- Unique user registration
- One-to-one messaging
- Message history retrieval and deletion
- User listing
- Offline delivery via an SQLite persistent store
- Graceful shutdown

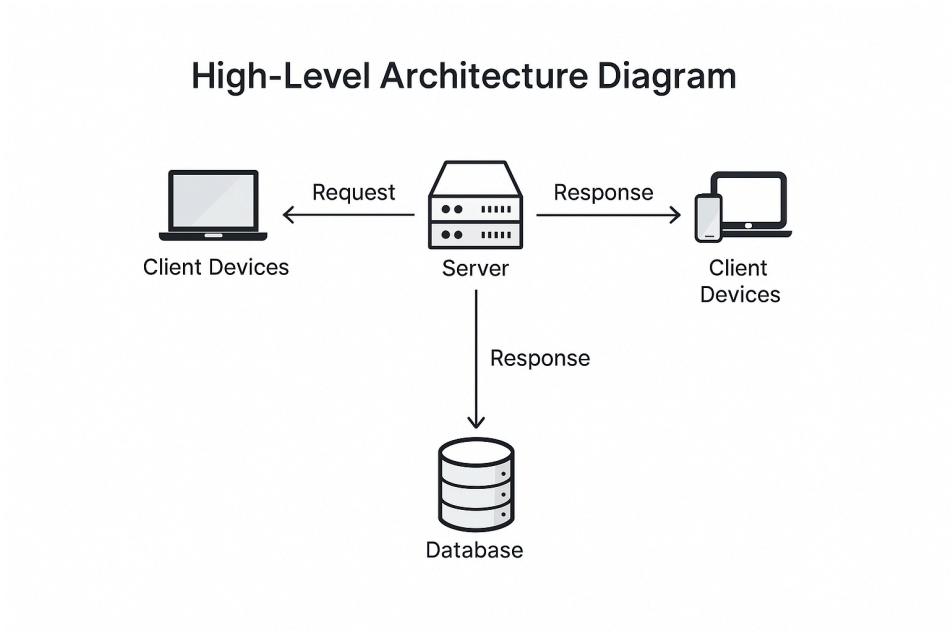
This design document focuses on:

- System architecture (client–server interaction, threads, and data flow)
- Server and client design, including their concurrency model
- Concurrency design (threading and synchronization of shared data)
- Testing strategy and unit tests

System Architecture



High-Level Architecture Diagram



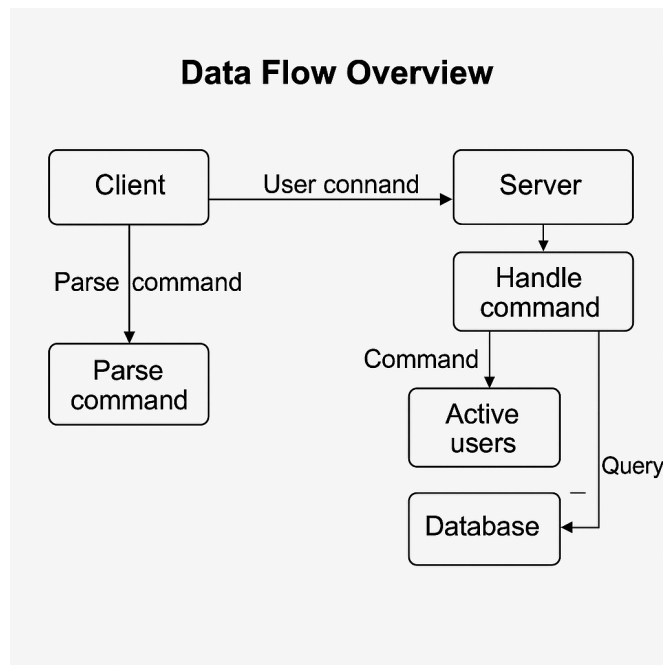
Description:

The server maintains a listening socket and spawns a **client handler thread** for each accepted connection.

Each client consists of:

- **Main thread** (interactive CLI, sending commands)
- **recv_thread** (listens asynchronously for server messages)

Data Flow Overview



Key logical flows:

Registration

- Client → USER | <username>
- Server validates and adds to active user list

- Server → OK | Registered or ERROR **Messaging**
- Client → SEND | receiver | message
- Server routes to online user or stores offline

History

- Client → GET | peer
- Server streams:
MSGHIST, multiple HIST lines, then ENDHIST

User Listing

- Client → USERS
- Server → USERLIST | <u1, u2, . . . >

Shutdown

- Server receives SIGINT
- Sets g_shutdown, closes listening socket
- Broadcasts SERVER | SHUTDOWN
- Cleans up users & DB

Thread & Responsibility Summary

Component	Thread	Main Responsibility
server.c	Main thread	Initialize DB/users, create listening socket, accept loop

server.c	client_thread	Handle one specific client's commands and lifecycle
handlers.c	Worker (in thread)	Parse SEND, GET, DELETE, USERS, EXIT
db.c	Internal	Synchronized DB operations (insert, query, delete)
users.c	Internal	Manage active user list and mapping username→socket
client.c	Main thread	Read CLI, validate input, send commands to server
client.c	recv_thread	Continuously recv() and print incoming server messages

Server Design

The server is implemented in **server.c**, with supporting modules:

- **db.c** – SQLite message store
- **users.c** – active user list
- **handlers.c** – protocol command parsing and processing

Server Startup & Initialization

Control flow (simplified):

main()

- |— signal(SIGINT, handle_sigint)
- |— init_db("messaging.db")
- |— init_users()
- |— socket()
- |— setsockopt(SO_REUSEADDR)
- |— bind()
- |— listen()
- └─ accept-loop (while !g_shutdown)
 - |— accept()
 - |— spawn pthread: client_thread()
 - └─ detach thread

- handle_sigint() sets g_shutdown and closes the listening socket for graceful termination.
server
- DB and user subsystem initialization happen **before** accepting clients.

Client Handler Thread (client_thread)

Each accepted connection is handled by client_thread(void *arg):

Registration Phase

1. recv() first line from client.
2. Expect "USER|<name>\n".
3. Validate:

- Not empty
 - Within max length
 - Characters: alphanumeric / _ / - only.
- server
4. Call `add_active_user(client_fd, username)`.
 - If duplicate username, send `ERROR|Username in use\n` and close.
 5. On success:
 - Log User '<name>' connected
 - Send `OK|Registered\n`
 - Call `push_undelivered_for_user(username, client_fd)` to send queued offline messages.

Command Handling Loop

SEND

- `SEND|receiver|message`
- Server stores or directly delivers
- Replies:
 - `OK|Delivered`
 - `OK|Stored`
 - `ERROR|Store failed`

GET

- MSGHIST header
- Server streams history using callback

- Ends with ENDHIST

DELETE

- Deletes history between two users

USERS

- Returns current active user list

EXIT

- Remove from active list
- Close connection
- Thread terminates

Unknown commands

- ERROR|Unknown command

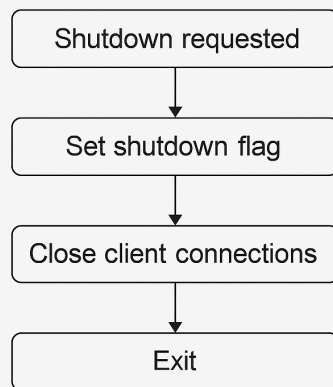
Disconnection Handling

If `recv()` returns ≤ 0 :

- Remove active user (by FD)
- Close socket
- Log disconnection

Server Shutdown Flow

Server Shutdown Flow



Shutdown Sequence:

1. User presses Ctrl+C
2. `handle_sigint()` sets `g_shutdown = 1`
3. Listening socket is closed
4. Accept loop exits
5. Server outputs “Shutting down server...”
6. Broadcast **SERVER | SHUTDOWN** to all clients
7. Shutdown users & close DB
8. Exit gracefully

This ensures all clients receive a **shutdown notice**, and resources are cleaned up in an orderly manner.

Key Server Responsibilities:

- Manage multiple clients concurrently
- Maintain DB integrity during reads/writes

- Implement secure authentication semantics
- Support message routing and retrieval

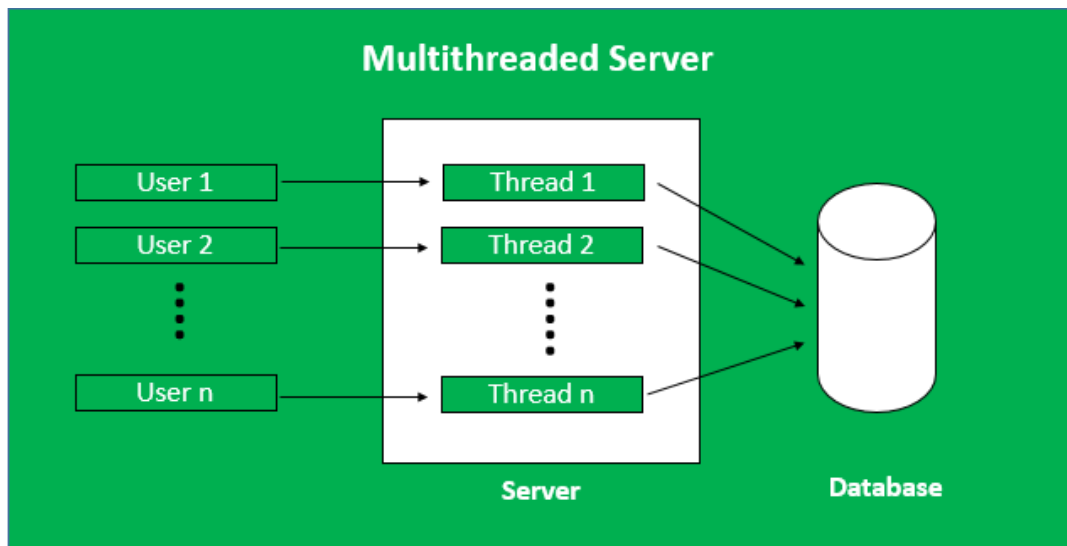
Client Design

The client implementation is in `client.c`. It is a simple CLI application with a **send loop** in the main thread and a **receive loop** in a background thread.

Client Startup

1. Validate command line
2. Create socket
3. Connect to server
4. Prompt for username → send `USER | <username>`
5. Wait for server response

Threading Model in Client (`recv_thread`)



main()

- |— set global sockfd & username
- |— pthread_create(recv_thread)
- └─ CLI loop (read & send commands)

recv_thread()

- └─ while (running)
 - |— recv(sockfd)
 - |— parse message
 - └─ print to console

- Only **main thread** calls send()
- Only **recv_thread** calls recv()
- Shared stop flag: `volatile int running`

Command Interface

Commands:

CLI Commands → Protocol Mapping

CLI Command	Protocol Sent
sendmessage u m	`SEND
getmessages u	`GET
deletemessages u	`DELETE
getuserlist	USERS

exit	EXIT
------	------

Messages recv_thread recognizes

- MSG | sender | message
- MSGHIST / HIST / ENDHIST
- USERLIST | . . .
- SERVER | SHUTDOWN

Concurrency Design

Server-Side Concurrency

Threading Model:

- **Thread-per-connection:** each client gets its own client_thread.

Shared Data:

1. **Active user list** (in users.c)
2. **SQLite database** (in db.c)
3. Global shutdown state g_shutdown
4. Listening file descriptor g_listen_fd

Synchronization Strategy (as per design & modules):

Shared Resource	Accessed By	Protection / Strategy
Active user list	All client threads, server main	Mutex in users.c (add/remove, lookup)
DB connections	All client threads via handlers	DB mutex in db.c wraps inserts/queries/deletes
Shutdown flag	Main thread, signal handler	volatile sig_atomic_t; accessed in loops
Listening socket	Accept loop & signal handler	Closed in signal handler to break blocking accept()

Why mutexes?

- Prevent race conditions when multiple threads:
 - Register users simultaneously
 - Send messages that require DB writes
 - Query message history concurrently.

Messaging System Design Document

- **Deadlock Avoidance:**

- Fine-grained locks inside modules (e.g., users vs. DB)
- Consistent lock acquisition order within handlers
- No nested circular dependencies.

Client-Side Concurrency

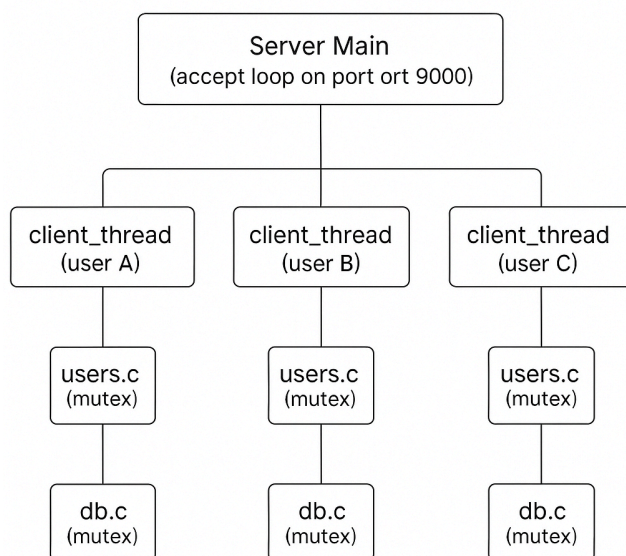
Shared State:

Variable	Threads Using It	Access Pattern
sockfd	main (send), recv_thread	Single writer for send, single reader for recv
running	main, recv_thread	Used as a stop flag; declared volatile
username	mostly main, read-only later	Written once at startup, then read-only

Given this pattern:

- No explicit mutex is needed for sockfd (no concurrent send/recv from multiple threads for the same direction).
- running uses volatile to reduce compiler reordering issues, and is checked in both loops.

Concurrency Flow Diagram (Server)



Each client thread **serializes** access to shared data through internal module mutexes, ensuring a thread-safe concurrent system.

Testing Strategy and Unit Tests

Testing is organized into four layers namely unit, integration, concurrency and stress testing.

Test Layers (Overview Chart)

Layer	Scope	Example Focus
Unit	Individual modules (db, users, handlers)	DB CRUD, user add/remove, parse commands
Integration	Client–server interaction end-to-end	Register, send, receive, history, delete
Concurren cy	Multiple clients, simultaneous operations	Race conditions, deadlock, correctness
Stress	Many clients & high message volume	Stability under load

Conclusion

This design document explains the messaging system architecture and its design. This implementation supports reliable real-time user communication and strictly follows the Operating System concepts such as multi-threading, synchronization, and process communication. Some future improvements such as secure user authentication, TLS-based encrypted communication, group messaging, presence tracking, message timestamps, and graphical interfaces could make this architecture more secure and user friendly.

In conclusion, this architecture shows a clear implementation of client-server architecture, multi-threaded programming and network protocol design.